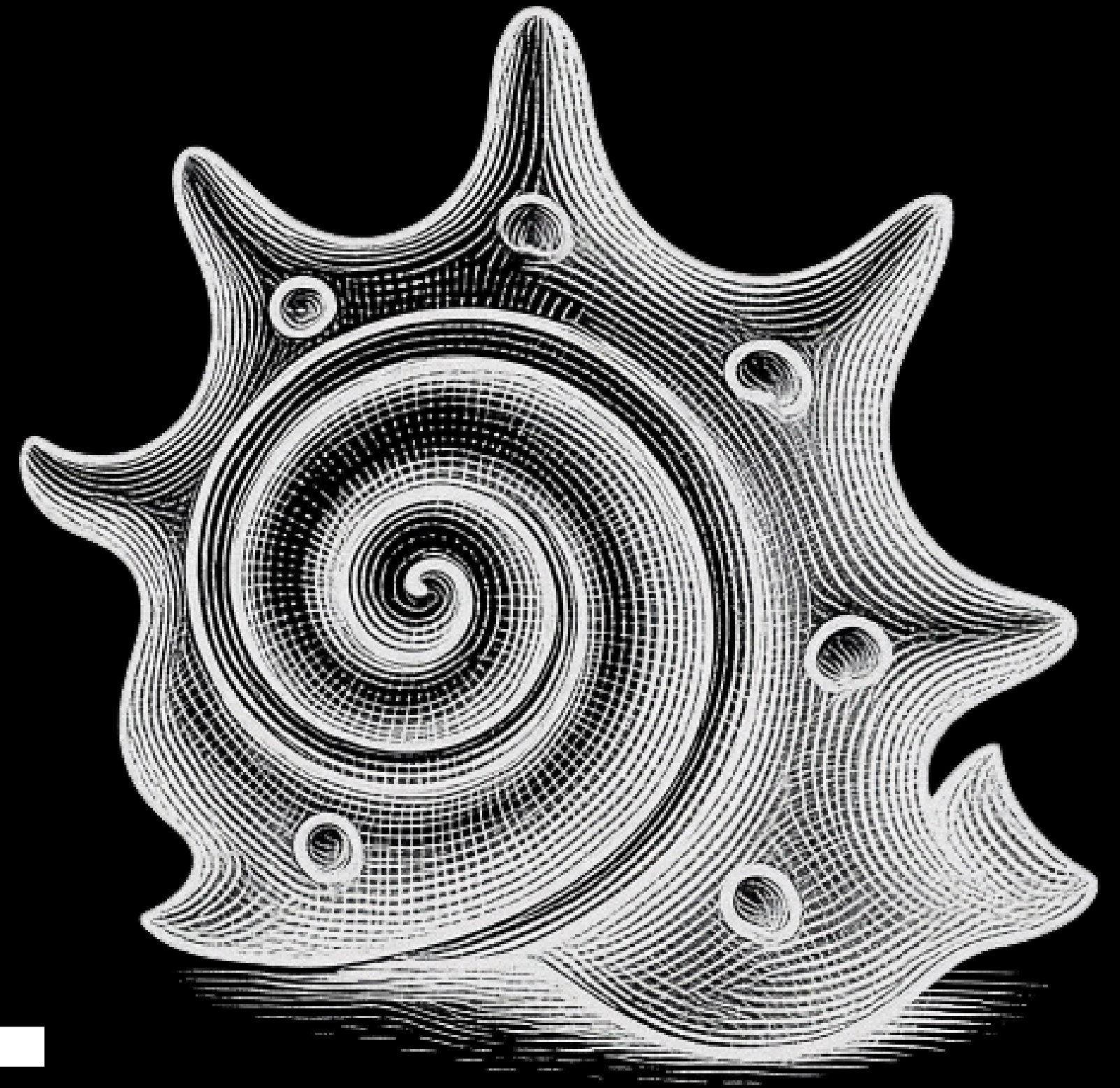


Data Science PBL SEM 3

PROJECT PROPOSAL



SEA SH

C based command line interpreter

As part of our Project-Based Learning (PBL) for Data Science in B.Sc. (Hons.) Computer Science (Semester 3), this project demonstrates the design and implementation of a custom Command-Line Interpreter developed purely in C language, integrated with predictive history search.

This work was developed under the guidance of our mentor,

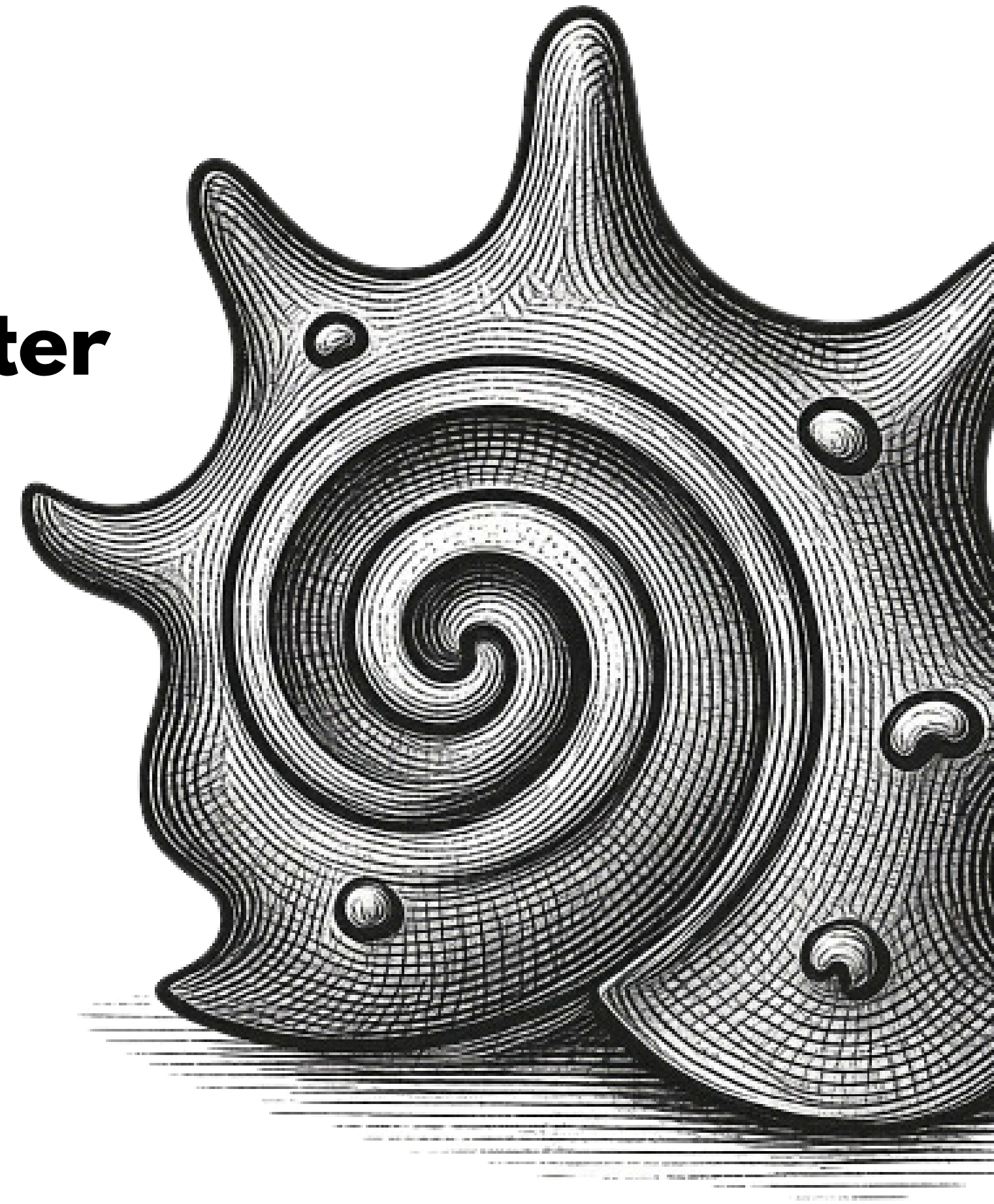
Prof. (Dr.) Varsha Mittal

Presented by:

Abhinav Bhatt (2511130001)

Sagar Singh (2511130006)

Benard Kigongo (25851004)



Introduction

- **Low level shell architecture** : A custom unix like cli to see how operating system manage processes, I/O and execution lifecycles.
- **Smart Terminal Experience** : Integrates interactive user-experience enhancements specifically real-time predictive text and command auto-completion directly into a minimal shell.
- **Core Focus** : Bridging low-level system calls (fork, exec, process piping) with efficient data structures for instant search and retrieval.

Problem Statement

- Modern shells (e.g., Zsh, Fish) offer powerful auto-suggestions, but their internal mechanics and system-level operations remain abstract and hidden from developers.
- Standard lightweight shells (e.g., vanilla Bash) rely on passive, rigid history lookup (history, Ctrl+R) rather than real-time, dynamic "ghost-text" suggestions as you type.
- To design and implement a lightweight shell in C that executes robust low-level process control while maintaining a data structure for real-time predictive history search.

Project Objectives

- **Core Shell Engine** : Implement a command line parser capable of process execution via POSIX system calls (fork, execvp, waitpid)
- **Dynamic History Traversal** : Build a memory backed history with keyboard event handling for seamless Up/Down arrow navigation.
- **Real-Time Predictive Search** : Develop a prefix search engine to display non-intrusive inline "ghost text" suggestions as the user types.
- **Smart Auto-Completion** : Enable Tab/Right Arrow to autocomplete suggestions directly into the active input line.

Literature Review

- **GNU Bash & Readline Library** : Standardizes UNIX process spawning and basic history access.
- **Fish & Zsh Autosuggestions** : Set the standard for developer productivity using real-time inline "ghost text" based on command frequency and recency.
- **Prefix Search & Trie Data Structures** : Academic standard for auto-complete systems. Storing history tokens in prefix trees enables instant lookups without slowing down terminal rendering.

Proposed Methodology

- **Dynamic History Storage** : Maintains in-memory linear buffers/prefix trees storing unique, successfully executed commands.
- **Realtime Prefix Matching** : With every keystroke, engine performs a reverse prefix scan across history to find the most recent matching command.
- **Ghost Text Rendering** : Outputs predicted text inline using ANSI terminal escape codes dim without corrupting current input.
- **State Transition** : Accepting Tab/Right Arrow copies the ghost prediction directly into the execution buffer.

System Architecture

- **Raw Input Interception** : Configure terminal to raw mode (termios.h) to capture keystrokes in real time rather than waiting for the Enter key.
- **Tokenization & Parsing** : Custom lexical analyzer breaks raw command strings into executable tokens.
- **Process Orchestration** :
 - Fork: Spawns a dedicated child process.
 - Exec: Replaces child process image.
 - Wait: Parent shell waits for process termination to prevent zombie processes.

Tools & Tech Used

- **Primary Language :**
 - Pure C for Low-level memory management and direct access to kernel primitives.
- **Operating System Primitives (POSIX APIs) :**
 - *unistd.h*, *sys/wait.h*, *sys/types.h* for process lifecycle (fork, execvp, waitpid).
 - *termios.h* for non canonical terminal input handling.
- **Build & Version Control :**
 - Git & GitHub: Branch-based team collaboration and source code control.

Development env.

- **Cross-Platform UNIX Development :**
 - **Native Linux (Ubuntu/Debian):** Primary testing environment for native POSIX system call execution.
 - **WSL 2 (Windows Subsystem for Linux) :** Full Linux kernel layer enabling identical POSIX compliance for team members developing on Windows.

This project is strictly created for unix os.

Expected Outcomes

- **Functional Standalone Shell Binary** : A compiled, zero-dependency C executable running natively on POSIX/Linux and WSL environments.
- **Interactive REPL with Ghost Suggestions** : Working terminal interface with real-time, prefix matching, arrow-key history traversal, and single key autocompletion.
- **Robust Process Lifecycle Control** : Clean execution of system commands without zombie processes.
- **Modular Source Codebase** : C codebase separating the parser, terminal raw-mode controller, process dispatcher, and history engine.

Future Scope

- **Piping & Redirection Support** : Implement interprocess communication through UNIX pipes (|) and standard file descriptor redirection (>, <).
- **Intelligent Ranking Algorithms** : Upgrading from linear prefix search to Trie-based frequency scoring (ranking suggestions by most frequently used commands).
- **Job Control & Background Tasks** : Adding signal handling (SIGINT, SIGTSTP) and background process management (&, fg, bg).

References

Advanced Programming in the UNIX Environment : Standard reference for process control.

Linux Programmer's Manual : for `termios(3)`, `unistd(2)`, and signal handling.

GNU C Library : Guidelines for standard string tokenization (`strtok/strsep`) and dynamic memory safety.

Fish / Zsh Project Technical Specs : Architecture references for inline autosuggestions.

THANK YOU